

Gabriel Lazzari Weiland

Desenvolvimento de um Sistema Distribuído (Web e Local) para Monitoramento e Gestão de Laboratórios de Pesquisa

Proposta de Trabalho de Conclusão de Curso apresentada ao Centro de Desenvolvimento Tecnológico da Universidade Federal de Pelotas, como requisito parcial à obtenção do título de Bacharel em Engenharia de Computação.

Orientador: Prof. Dr. Luciano Volcan Agostini
Coorientadores: Prof. Dr. Ruhan Avila da Conceição
M.Sc. Gustavo Hübner Rehbein

Pelotas, 2026

1 DADOS DE IDENTIFICAÇÃO

1.1 Nome do Projeto

Desenvolvimento de um Sistema Distribuído(Web e Local) para Monitoramento e Gestão de Laboratórios de Pesquisa

1.2 Local de Realização

Laboratório do ViTech / Centro de Desenvolvimento Tecnológico (CDTec) / Universidade Federal de Pelotas (UFPel)

1.3 Responsável pelo Projeto

Gabriel Lazzari Weiand
glweiand@inf.ufpel.edu.br

1.4 Professor Orientador

Prof. Dr. Luciano Volcan Agostini

1.5 Professor Co-orientador

Prof. Me. Ruhan Ávila da Conceição
M.Sc. Gustavo Hübner Rehbein

2 SUMÁRIO EXECUTIVO

Partimos do problema no gerenciamento do uso dos recursos computacionais de um laboratório de pesquisa computacional, como o Video Technology Research Group (ViTech), que atualmente depende de métodos informais para o agendamento do uso dos PCs como trocas de mensagens e o uso de planilhas para gerenciar estes processos. Este cenário é desfavorável tanto para a organização e o uso eficiente dos equipamentos como para a segurança dos recursos de alto valor do laboratório.

Assim sendo, a proposta deste Trabalho de Conclusão de Curso (TCC) é desenvolver um software robusto, centralizado e seguro. A solução a ser desenvolvida consiste num sistema distribuído (Web e local) que formaliza todo esse ciclo de vida. Essa aplicação tem objetivo de atender diferentes perfis de usuários: Estudantes, que poderão fazer e visualizar seus agendamentos; Administradores, responsáveis pela alocação ou não das máquinas aos usuários e administração geral do sistema. O objetivo final é prover uma ferramenta que gere eficiência, segurança e uma trilha de auditoria completa do uso do laboratório.

Com isso, é previsto um melhor e mais eficiente uso de recursos computacionais, tornando mais conveniente tanto para os usuários quanto para os administradores o aproveitamento e o gerenciamento de um laboratório de pesquisa com foco em computação, assim como tornar possível a detecção de usos ineficientes e indevidos. Também é um fundamento do projeto seguir as melhores práticas aprendidas ao longo da jornada acadêmica para que além de robusto e seguro, esse projeto seja igualmente modular, de fácil manutenção e altamente intuitivo para os usuários.

3 HISTÓRICO E JUSTIFICATIVA

Ao decorrer de um curso de computação o graduando passa grande parte do seu tempo em laboratórios de informática, seja em aulas ou em pesquisas. Considerando a alta demanda e, em muitos casos, a escassez de recursos, surge a ideia de desenvolver um software para otimizar esse gerenciamento. Baseado no esquema atual de laboratórios de pesquisa como o do ViTech, encontramos um sistema fundamentado em conversas diretas com os responsáveis e no armazenamento em planilhas. A ideia deste trabalho é solucionar não apenas o problema da ineficiência na obtenção de acesso às máquinas para uso, mas também garantir que esses recursos estejam realmente dedicados ao usuário e que este faça um uso adequado das ferramentas.

O problema tratado é um exemplo típico da necessidade de migração de um modelo centralizado e informal para um sistema distribuído. A arquitetura proposta neste trabalho, composta por servidor web, servidor de controle local e agentes nas máquinas, se enquadra perfeitamente na definição de um Sistema Distribuído (SD): "Uma coleção de computadores independentes que se apresenta aos seus usuários como um sistema único e coerente" (?). A principal justificativa para esta abordagem é que "o objetivo principal de um sistema distribuído é tornar mais fácil para os usuários (e aplicações) acessarem recursos remotos e compartilhá-los de maneira controlada e eficiente" (?). O sistema atual falha exatamente nesse controle, enquanto a solução proposta visa implementar uma solução para o mesmo.

Além da necessidade de prover um melhor controle, a migração para uma solução de software justifica-se por alguns objetivos centrais que o sistema atual de planilhas não é capaz de oferecer: confiabilidade, escalabilidade e manutenibilidade. O modelo atual, dependente da comunicação informal e da edição manual de arquivos, por natureza é frágil e suscetível a erro humano, falhando em ser confiável. A confiabilidade em um sistema significa garantir que ele continue a funcionar corretamente, mesmo diante de imprevistos, como falhas de hardware, software ou intervenção humana incorreta (Kleppmann, 2017). A introdução de um sistema assistido para o agendamento e de um controle de acesso automatizado é a forma escolhida de garantir que o estado desejado, conforme o agendamento, seja consistentemente aplicado ao estado

real do laboratório.

A chave para alcançar a robustez neste projeto é a implementação da separação de interesses. Como afirma Robert C. Martin, uma boa arquitetura de software atinge esse objetivo "traçando linhas entre esses interesses. Essas linhas são os limites" (Martin, 2019). Ao estabelecer limites arquiteturais claros e isolar a interface de usuário da lógica de controle e dos mecanismos de máquina, o sistema ganha alta testabilidade e independência de seus componentes. A escolha das tecnologias a serem utilizadas parte desse fundamento principalmente pela necessidade de robustez e clareza.

Finalmente, o projeto se justifica pela incapacidade dos modelos informais atuais de escalar. A escalabilidade é "o termo que usamos para descrever a capacidade de um sistema de lidar com o aumento da carga" (Kleppmann, 2017). O gerenciamento, sendo feito de forma manual, torna-se desnecessariamente mais complexo e lento a cada novo usuário, máquina ou regra de agendamento adicionada. A arquitetura proposta é projetada com foco na extensibilidade, permitindo que novos componentes, máquinas ou alunos, sejam adicionados à rede sem gerar desgaste tanto para os usuários como para o sistema, de forma que justifique o investimento em desenvolver uma arquitetura limpa e bem definida.

4 OBJETIVOS E METAS

4.1 Objetivo Geral

O objetivo geral deste projeto é desenvolver uma solução para o gerenciamento de recursos computacionais de um laboratório de pesquisa, fazendo isso através do projeto, implementação e validação de um sistema distribuído que irá, desde a parte da alocação dos recursos via Web até a garantia de que o estado desejado do laboratório seja atingido no mundo real.

Este sistema visa substituir o processo de agendamento de máquinas que atualmente ocorre de forma informal e manual por uma solução assistida por máquina na alocação e completamente automatizada na validação do uso real dos computadores, provendo eficiência no processo de alocação e uso para todos os tipos de usuários (Estudantes e Administradores).

4.2 Objetivos específicos

4.2.1 Definição de Requisitos

Definir os requisitos do sistema, funcionais e não-funcionais, com base no problema atual do laboratório e na análise da literatura sobre arquiteturas distribuídas, autenticação e sistemas de *logs*.

4.2.2 Projeto da Arquitetura

Projetar a arquitetura de três componentes (Servidor de Alocação Web, Servidor de Controle Local e Agente de Máquina), detalhando os modelos de dados, as *Application Programming Interfaces* (APIs) e a estratégia de segurança e separação de interesses.

4.2.3 Implementação do Servidor de Alocação (Web)

Implementar o *backend*, através de uma API com princípios de arquitetura *Representational State Transfer* (REST) com AdonisJS, para gerenciar o estado desejado e o *frontend* para interação dos usuários (agendamento e gerenciamento).

4.2.4 Desenvolvimento do Servidor de Controle (Local)

Desenvolver o servidor de Controle (Local), implementando a lógica de *pooling* para uma comunicação segura com o servidor Web e banco de dados (SQLite) para armazenar o estado real do laboratório.

4.2.5 Implementação do Agente de Máquina

Implementar o agente de máquina, capaz de executar as duas principais funções planejadas: o gerenciamento de acesso do usuário (bloqueio/liberação da máquina) e o monitoramento de métricas de hardware (CPU, GPU, memória) e uso.

4.2.6 Projeto e Implementação do Sistema de Logs e Auditoria

Projetar e implementar um sistema de *logs* e auditoria, de forma que, todas as ações de agendamento, acesso e monitoramento sejam registradas de forma confiável e segura no Servidor de Controle.

4.2.7 Validação do projeto

Validar o projeto em um ambiente real (Laboratório ViTech), avaliando a performance, usabilidade e segurança do sistema com base nos *feedbacks* dos usuários.

5 METODOLOGIA

Em uma arquitetura como a proposta, o sistema de tipos estáticos é crucial para garantir a integridade dos contratos de dados entre o Servidor Web, o Servidor de Controle e os Agentes, prevenindo erros de tempo de execução que poderiam comprometer o funcionamento do sistema. Sendo assim, a linguagem TypeScript foi escolhida como base para os componentes por ser um *superset* do JavaScript que "permite que os desenvolvedores usem tipos estáticos para detectar *bugs* em tempo de desenvolvimento, antes que o código seja executado" (Microsoft, 2025).

A linguagem foi projetada para "auxiliar no desenvolvimento de códigos simples até os mais complexos, utilizando os princípios de Orientação a Objetos, como classes, tipagens, interfaces, *Generics* entre outros", e sua "utilização de tipagem ajuda no momento de depuração (*debug*) do código, prevenindo assim possíveis *bugs* ainda em tempo de desenvolvimento" (Silva adriano, 2021).

No desenvolvimento do Servidor de Alocação (Web), o *framework* AdonisJS foi selecionado por ter foco em "ergonomia do desenvolvedor, estabilidade e velocidade"(Team, 2025). Ele fornece uma estrutura *backend* robusta e segura para a construção da API REST que gerencia o "estado desejado", permitindo que o desenvolvimento se concentre na lógica de negócios (perfis de usuário, regras de agendamento) em vez da configuração de baixo nível. Por fim, a escolha do SQLite como banco de dados é uma decisão arquitetural que reforça o requisito de segurança e isolamento. O SQLite é "um motor de banco de dados *Structured Query Language* (SQL) transacional completo" que assegura as propriedades de Atomicidade, Consistência, Isolamento e Durabilidade (ACID) (Hipp, 2025). Isso garante que o "cérebro" da operação possa manter o estado das alocações, a trilha de auditoria e os dados de monitoramento de forma confiável, diretamente acoplado à aplicação de controle, sem introduzir a complexidade e o ponto de falha de um servidor de banco de dados de rede.

6 PLANO DE ATIVIDADES E CRONOGRAMA

As atividades, e seus respectivos prazos, desse plano foram planejados para serem realizadas no ano disponível para a execução e conclusão do trabalho de acordo com a descrição e a tabela a seguir:

Atividade 1 - Refinamento do escopo do problema e objetivos.

Atividade 2 - Revisão bibliográfica (Arquiteturas Distribuídas, APIs, Logs, Autenticação).

Atividade 3 - Levantamento e definição de requisitos (Funcionais e Não-Funcionais).

Atividade 4 - Projeto da arquitetura (3 componentes, APIs, segurança, bancos de dados).

Atividade 5 - Implementação do Servidor de Alocação (Web) - *Backend* (AdonisJS API REST).

Atividade 6 - Escrita e entrega da monografia parcial.

Atividade 7 - Implementação do Servidor de Controle (Local) - Lógica de *pooling* e banco de dados.

Atividade 8 - Implementação do Servidor de Alocação (Web) - *Frontend* (Interface de usuário).

Atividade 9 - Projeto e implementação do sistema de *logs* e auditoria (Coleta e armazenamento).

Atividade 10 - Implementação do Agente de Máquina (Módulo de monitoramento: CPU, GPU, entre outras métricas).

Atividade 11 - Implantação e testes em ambiente real.

Atividade 12 - Identificação e correção de bugs com base nos testes.

Atividade 13 - Refinamento da Interface do Usuário (UI) e Experiência de Uso (UX) com base nos *feedbacks*.

Atividade 14 - Escrita do texto final.

Atividade 15 - Entrega do texto final e defesa do TCC.

REFERÊNCIAS

HIPP, D. R. e. a. **SQLite: About**. 2025. Disponível em: <<https://www.sqlite.org/about.html>>. Acesso em: 2025-11-05.

KLEPPMANN, M. **Designing Data-Intensive Applications**: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. Sebastopol, CA, USA: O'Reilly Media, 2017.

MARTIN, R. C. **Arquitetura Limpa**: O Guia do Artesão para Estrutura e Design de Software. Rio de Janeiro, RJ: Alta Books, 2019. Tradução de: Clean Architecture: A Craftsman's Guide to Software Structure and Design.

MICROSOFT. **TypeScript**: JavaScript com Sintaxe para Tipos. 2025. Disponível em: <<https://www.typescriptlang.org/>>. Acesso em: 2025-11-05.

SILVA ADRIANO, T. da. **Guia prático de TypeScript**: Melhore suas aplicações JavaScript. São Paulo, SP, Brazil: Casa do Código, 2021.

TEAM, A. **AdonisJS - A Node.js framework focused on developer ergonomics**. 2025. Disponível em: <<https://adonisjs.com/>>. Acesso em: 2025-11-05.

7 ASSINATURAS

Gabriel Lazzari Weiand
Proponente

Luciano Volcan Agostini
Prof. Orientador